CSS IV — Interacciones, animaciones, scroll y puente a Tailwind CSS

En esta sesión

En esta última sesión del bloque de CSS vas a centrarte en una parte muy visible de las interfaces modernas: cómo responden visualmente a la interacción, cómo se animan con criterio y cómo se conectan con una herramienta actual como Tailwind CSS v4.

Vas a estudiar:

- estados interactivos;
- transiciones;
- transformaciones;
- animaciones con `@keyframes`;
- animaciones basadas en scroll;
- organización final del CSS;
- y el papel de Tailwind CSS v4 como herramienta utility-first.

---

1. Introducción

Hasta aquí ya has trabajado estructura visual, box model, Flexbox, Grid, `position`, imágenes y responsive. Con eso puedes construir interfaces bastante completas. Pero una interfaz bien hecha no solo necesita verse correctamente: también debe **reaccionar con claridad** cuando una persona usuaria pasa el cursor, recibe foco, pulsa un botón o se desplaza por la página. CSS ofrece herramientas distintas para cada uno de esos comportamientos: transiciones para cambios entre estados, transformaciones para modificar visualmente un elemento y animaciones para secuencias más completas.

Idea clave

Una buena interacción visual no está para “decorar”.
Está para comunicar mejor qué puede hacerse, qué está ocurriendo y qué acaba de cambiar.

En esta sesión también aparece una idea importante de cierre: escribir CSS no consiste solo en conocer propiedades, sino en **usar las adecuadas con moderación** y mantener el archivo organizado. Por eso esta sesión combina comportamiento visual con mantenimiento y termina enlazando con Tailwind CSS v4, que se presenta oficialmente como un framework utility-first para construir interfaces modernas.

Referencias del apartado 1

- MDN Web Docs. *Using CSS transitions*.
- MDN Web Docs. *Using CSS animations*.
- Tailwind CSS. *Tailwind CSS v4.0*.
- Tailwind CSS. *Theme variables*.

---

2. Estados interactivos y transiciones

Una **transición** sirve para suavizar el paso entre dos estados visuales. MDN explica que una transición define el cambio entre un estado inicial y otro final, y que suele activarse por pseudo-clases como `:hover` o `:active`, o mediante cambios aplicados dinámicamente. Esa es la pista más importante para entender cuándo usarla: si el cambio depende de un estado, normalmente debes pensar primero en **transición**.

Ejemplo básico:

```
.button {
  background-color: #2563eb;
  transition: background-color 0.25s ease;
}

.button:hover {
  background-color: #1d4ed8;
}
```

Las transiciones suelen apoyarse en estados como estos:

- `:hover`
- `:focus`
- `:focus-visible`
- `:active`

Eso es importante porque una transición no tiene sentido por sí sola: necesita un cambio de estado real. En una interfaz bien planteada, ese cambio puede ayudar a entender que un

elemento es interactivo, que acaba de recibir foco o que está siendo pulsado.

Tiempos en transiciones

La propiedad `transition` resume varias subpropiedades: qué propiedad cambia, cuánto dura el cambio, qué curva temporal sigue y si existe retraso. En la práctica, eso significa que el tiempo también comunica. Duraciones cortas suelen funcionar mejor en microinteracciones; duraciones largas, en cambio, pueden hacer que la interfaz parezca pesada.

Ejemplo con varios parámetros:

```
.link {
  transition: color 0.2s ease, text-decoration-color 0.2s ease;
}
```

Buena práctica

Si personalizas un estado de foco, no elimines su visibilidad.
Una interfaz clara también debe ser usable con teclado.

Cuándo encaja mejor una transición

Una transición suele tener más sentido en:

- cambios de color o fondo;
- hover en botones y enlaces;
- pequeñas variaciones de sombra;
- foco visible en inputs o botones;
- cambios de escala o desplazamientos muy cortos.

Referencias del apartado 2

- MDN Web Docs. *Using CSS transitions*.
- MDN Web Docs. *transition*.
- MDN Web Docs. *CSS transitions*.

---

3. Transformaciones

Las transformaciones modifican visualmente un elemento sin rehacer el layout como lo harían otras propiedades. MDN resume `transform` como la propiedad que permite trasladar, rotar,

escalar o deformar un elemento. En interfaces reales, las transformaciones suelen combinarse con transiciones para construir microinteracciones claras.

Las más útiles al empezar suelen ser:

- `translate`
- `scale`
- `rotate`

Ejemplos:

```
.card:hover {
  transform: translateY(-4px);
}

.button:active {
  transform: scale(0.98);
}
```

Esto aparece con frecuencia en:

- botones;
- tarjetas;
- iconos;
- pequeños indicadores de interacción;
- o bloques que necesitan una respuesta visual discreta.

> **Regla práctica**
>
> Si un elemento ya cambia de estado con `:hover`, `:focus` o `:active`, muchas veces una pequeña transformación combinada con transición es suficiente.
> No hace falta convertir cada interacción en una animación compleja.

Referencias del apartado 3

- MDN Web Docs. *transform*.
- MDN Web Docs. *CSS transforms*.

---

4. Animaciones con `@keyframes`

Una **animación** no se limita a un cambio entre estado A y estado B. MDN explica que las animaciones CSS permiten pasar de una configuración visual a otra mediante una definición de animación y un conjunto de `@keyframes` que incluyen estados iniciales, finales y, si hace falta, puntos intermedios. Esa es la diferencia esencial respecto a una transición: aquí ya no dependes solo de un estado, sino de una **secuencia temporal definida**.

Ejemplo:

```
@keyframes fade-in-up {
  from {
    opacity: 0;
    transform: translateY(16px);
  }

  to {
    opacity: 1;
    transform: translateY(0);
  }
}

.notice {
  animation: fade-in-up 0.4s ease both;
}
```

La shorthand `animation` agrupa varias subpropiedades como:

- `animation-name`
- `animation-duration`
- `animation-timing-function`
- `animation-delay`
- `animation-iteration-count`
- `animation-fill-mode`
- `animation-direction`

Eso significa que, igual que en las transiciones, el tiempo también importa aquí, pero con más matices: puedes decidir retrasos, repeticiones, persistencia del estado final o dirección de reproducción.

Cuándo encaja mejor una animación

Una animación suele tener más sentido en:

- entradas en pantalla;

- apariciones progresivas;
- mensajes o avisos;
- loaders;
- o secuencias visuales que no dependen solo de un hover o un focus.

> ⚠ **Buen criterio**
>
> Una animación útil aclara una transición de estado o una entrada en pantalla.
> Una animación gratuita distrae y suele empeorar la interfaz.

Diferencia práctica entre transición y animación

Recurso	Cuándo encaja mejor
Transición	Cambio entre dos estados: `hover`, `focus`, `active`, color, sombra, escala, desplazamiento corto
Animación	Secuencia temporal definida: entrada, aviso, loader, repetición controlada, aparición progresiva

Referencias del apartado 4

- MDN Web Docs. *Using CSS animations*.
- MDN Web Docs. *animation*.

---

5. Animaciones basadas en scroll

El CSS moderno permite animaciones ligadas al desplazamiento sin necesidad de JavaScript. MDN explica que las **scroll-driven animations** hacen progresar una animación según una línea temporal basada en scroll, en lugar de la línea temporal tradicional basada solo en tiempo. Eso permite animar un elemento al desplazarse la página o cuando el elemento entra en el área visible de un contenedor.

Hay dos ideas principales aquí:

- **scroll progress timeline**: la animación avanza según cuánto se ha desplazado un contenedor;
- **view progress timeline**: la animación avanza según cuánto entra o sale un elemento del área visible.

Ejemplo conceptual:

```
@keyframes reveal {
  from {
    opacity: 0;
    transform: translateY(24px);
  }

  to {
    opacity: 1;
    transform: translateY(0);
  }
}

.card {
  animation: reveal linear both;
  animation-timeline: view();
  animation-range: entry 0% cover 40%;
}
```

MDN documenta funciones como `scroll()` y `view()` para trabajar con `animation-timeline`, y también aclara una particularidad importante: si usas la shorthand `animation`, debes declarar `animation-timeline` después, porque la shorthand la reinicia a `auto`. Además, cuando usas `scroll-timeline`, la duración deja de depender del tiempo y pasa a depender del avance del scroll.

> ⚠ **Compatibilidad**
>
> Las scroll-driven animations siguen teniendo disponibilidad limitada en algunos navegadores, así que conviene tratarlas como un contenido moderno y valioso para explorar, pero no como una base obligatoria para cualquier proyecto de producción.

Eso no les quita interés didáctico. Al contrario: sirven muy bien para mostrar hacia dónde evoluciona CSS y cómo ciertas animaciones antes ligadas a JavaScript pueden resolverse ahora desde el propio lenguaje de estilos. Aun así, el criterio sigue siendo el mismo: solo tienen sentido si mejoran la lectura visual del contenido y no convierten la página en una sucesión de efectos.

Referencias del apartado 5

- MDN Web Docs. *CSS scroll-driven animations*.
- MDN Web Docs. *Scroll-driven animation timelines*.

- MDN Web Docs. *animation-timeline*.
- MDN Web Docs. *scroll()*.
- MDN Web Docs. *scroll-timeline*.

---

6. Organización del CSS para interacciones y efectos

A estas alturas del bloque ya no basta con que el CSS “funcione”. También importa mucho cómo se organiza para poder mantenerlo y ampliarlo después.

Una forma razonable de pensar el archivo de estilos es distinguir bloques como estos:

Bloque	Qué suele incluir
Base	`body`, tipografía, colores globales, resets ligeros
Layout	contenedores, cabeceras, grids generales
Componentes	botones, tarjetas, formularios, badges
Estados e interacción	`:hover`, `:focus-visible`, `:active`, transiciones
Animaciones	`@keyframes`, reglas específicas de animación, scroll-driven animations

No se trata de imponer una estructura universal, sino de evitar que el archivo crezca como una lista desordenada de reglas sin relación.

> **Idea práctica**
>
> No refactorizas CSS solo cuando “se rompe”.
> También lo refactorizas cuando empiezas a notar duplicaciones, selectores demasiado largos, reglas que se pisan o bloques difíciles de localizar.

En este punto también cobra más valor lo ya trabajado anteriormente:

- las **variables CSS** ayudan a sostener coherencia;
- **BEM** ayuda a nombrar mejor las piezas;
- **nesting** puede mejorar lectura si no se abusa;
- y **Stylelint** ayuda a mantener consistencia.

7. Tailwind CSS v4

Tailwind CSS se presenta oficialmente como un framework **utility-first** para construir interfaces directamente desde utilidades pequeñas combinadas en el marcado. Eso significa que muchas decisiones visuales que en CSS tradicional escribirías como reglas completas pasan a expresarse mediante clases utilitarias.

Qué problema resuelve

Tailwind puede aportar:

- rapidez al maquetar;
- consistencia visual;
- menor repetición de CSS manual;
- y una relación más directa con spacing, color, tipografía, layout y estados.

Pero no decide por ti:

- la jerarquía visual;
- el layout correcto;
- qué interacción tiene sentido;
- ni qué animación es adecuada.

> **Idea clave**
>
> Tailwind no sustituye el criterio.
> Solo cambia la forma de expresar decisiones que sigues teniendo que tomar tú.

Cómo conecta con todo lo aprendido

Lo aprendido en CSS	Cómo reaparece en Tailwind
`margin`, `padding`, `gap`	utilidades de spacing
color y fondos	utilidades de color
tipografía	utilidades de texto
Flexbox y Grid	utilidades de layout
responsive	variantes por breakpoint

Lo aprendido en CSS	Cómo reaparece en Tailwind
`hover`, `focus`, `active`	variantes de estado
transiciones y animaciones	utilidades de transición y animación
variables y tokens	`@theme` y theme variables

Qué cambia en v4

Tailwind CSS v4 introdujo una reorganización importante del framework. Su lanzamiento oficial destaca una experiencia de configuración y personalización rediseñada. La documentación actual explica que los **theme variables** se definen con `@theme`, una directiva que influye en las utilidades disponibles en el proyecto.

Ejemplo conceptual:

```
@import "tailwindcss";

@theme {
  --color-mint-500: oklch(0.72 0.11 178);
}
```

A partir de ahí, Tailwind genera utilidades relacionadas con ese token, como clases de fondo o de texto para ese color. Esto conecta muy bien con una idea ya trabajada en CSS nativo: pensar en **tokens**, variables y consistencia visual.

Qué conviene saber de v4

La documentación oficial también deja claras varias ideas prácticas:

- el responsive sigue siendo central y se articula mediante variantes por breakpoint;
- v4 está diseñado para navegadores modernos; si necesitas soportar navegadores antiguos, la propia guía recomienda seguir en v3.4;
- y v4 no está pensado para combinarse con preprocesadores como Sass, Less o Stylus.

Tailwind también permite salirte puntualmente de las utilidades predefinidas con estilos arbitrarios o personalizados cuando hace falta, pero eso no invalida la idea central del framework: trabajar con un sistema de utilidades y tokens consistente.

Por qué tiene sentido verlo al final

Si ya entiendes:

- box model;
- Flexbox;
- Grid;
- responsive;
- estados interactivos;
- transiciones;
- animaciones;
- y organización del CSS;

entonces Tailwind deja de ser una colección de clases misteriosas y pasa a convertirse en una herramienta bastante más comprensible.

Referencias del apartado 7

- Tailwind CSS. *Tailwind CSS v4.0*.
- Tailwind CSS. *Theme variables*.
- Tailwind CSS. *Upgrade guide*.
- Tailwind CSS. *Adding custom styles*.
- Tailwind CSS. *Dark mode*.

---

8. Análisis técnico avanzado y cierre

Esta sesión cierra el bloque de CSS con una idea central: una interfaz no está realmente bien resuelta solo porque tenga color, layout y tipografía. También debe responder con claridad a la interacción, usar el movimiento con criterio y poder mantenerse cuando el proyecto crece.

Los errores más habituales en esta fase no suelen venir de una propiedad concreta, sino de decisiones de enfoque:

- transiciones puestas “porque sí”;
- hover llamativos que no aportan información real;
- foco visual pobre o directamente eliminado;
- animaciones largas que ralentizan la lectura;
- efectos de scroll introducidos sin pensar en compatibilidad ni utilidad;
- o interés por Tailwind como atajo, sin entender antes el CSS que hay debajo.

Lo verdaderamente profesional aquí no es acumular efectos, sino elegir bien:

- qué interacción debe notarse;

- qué cambio merece una transición;
- qué elemento necesita una transformación;
- qué animación aporta claridad;
- y qué parte del sistema debe mantenerse simple.

Qué debes retener de esta sesión

- Una **transición** suaviza un cambio entre estados.
- Una **animación** define una secuencia temporal más completa.
- Las **transformaciones** son una herramienta visual muy útil en microinteracciones.
- Las **scroll-driven animations** son una posibilidad moderna interesante, pero todavía deben tratarse con cautela por compatibilidad.
- Una buena interfaz usa el movimiento con moderación y propósito.
- El CSS también debe organizarse para poder mantenerse.
- Tailwind CSS v4 es una herramienta utility-first moderna que se entiende mucho mejor cuando ya dominas CSS nativo.

Referencias documentales generales

1. MDN Web Docs. *Using CSS transitions*.
2. MDN Web Docs. *transition*.
3. MDN Web Docs. *Using CSS animations*.
4. MDN Web Docs. *animation*.
5. MDN Web Docs. *CSS scroll-driven animations*.
6. MDN Web Docs. *Scroll-driven animation timelines*.
7. MDN Web Docs. *animation-timeline*.
8. Tailwind CSS. *Tailwind CSS v4.0*.
9. Tailwind CSS. *Theme variables*.
10. Tailwind CSS. *Upgrade guide*.

Mini glosario

Término	Definición breve
Transición	Cambio gradual entre dos estados visuales
Transformación	Modificación visual como mover, rotar o escalar
Animación	Secuencia definida con `@keyframes`

Término	Definición breve
Timeline	Línea temporal que hace progresar una animación
Scroll-driven animation	Animación cuyo progreso depende del desplazamiento
`animation-timeline`	Propiedad que define la línea temporal de una animación
Utility-first	Enfoque basado en utilidades pequeñas reutilizables
`@theme`	Directiva de Tailwind v4 para definir variables de tema

Fragmento de ejemplo integrador

HTML

```html
<section class="course-grid">
  <article class="course-card">
    <h2>HTML y CSS</h2>
    <p>Aprende a construir interfaces limpias, claras y mantenibles.</p>
    <a class="course-link" href="#">Ver curso</a>
  </article>

  <article class="course-card">
    <h2>JavaScript</h2>
    <p>Empieza a añadir comportamiento real a tus interfaces.</p>
    <a class="course-link" href="#">Ver curso</a>
  </article>

  <article class="course-card">
    <h2>React</h2>
    <p>Construye interfaces modernas basadas en componentes.</p>
    <a class="course-link" href="#">Ver curso</a>
  </article>
</section>
```

CSS

```css
.course-grid {
  display: grid;
  gap: 16px;
}

.course-card {
  padding: 16px;
```

```
  border-radius: 12px;
  background-color: white;
  box-shadow: 0 8px 24px rgba(0, 0, 0, 0.08);
  transition: transform 0.25s ease, box-shadow 0.25s ease;
}

.course-card:hover {
  transform: translateY(-4px);
  box-shadow: 0 12px 28px rgba(0, 0, 0, 0.12);
}

.course-link {
  text-decoration: none;
}

.course-link:hover,
.course-link:focus-visible {
  text-decoration: underline;
}

@keyframes fade-in-up {
  from {
    opacity: 0;
    transform: translateY(16px);
  }

  to {
    opacity: 1;
    transform: translateY(0);
  }
}

.course-card {
  animation: fade-in-up 0.4s ease both;
}

/* Ejemplo conceptual de scroll-driven animation */
/*
.course-card {
  animation: fade-in-up linear both;
  animation-timeline: view();
  animation-range: entry 0% cover 35%;
}
*/
```